



ЗВІТ

Лабораторна робота № 5.1

Дисципліна: Базы даних та інформаційні системи

Тема: MongoDB: базові операції, запити та індексація

Виконав

Студент 3 курсу

Групи МІТ-31

Факультету інформаційних технологій

Бабяк Володимир

Мета

Ознайомитися з MongoDB як документоорієнтованою NoSQL БД. Навчитися створювати базу даних і колекції, виконувати CRUD-операції, будувати запити з операторами фільтрації та створювати індекси. Підготувати набір даних для виконання лабораторної роботи № 5.2 (Aggregation Framework).

Хід роботи

Варіант 3: Доставка їжі

New Connection

×

Manage your connection settings

URI ⓘ

Edit Connection String ☐

mongodb://localhost:27017/

Name

lab5

Color

No Color ▼

☒ **Favorite this connection**
Favoriting a connection will pin it to the top of your list of connections

➤ Advanced Connection Options

Cancel

Save

Connect

Save & Connect

How do I find my connection string in Atlas?

If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.

[See example ↗](#)

How do I format my connection string?

[See example ↗](#)

1. Створення бази даних та колекцій

На початковому етапі було створено базу даних lab5 та три колекції відповідно до Варіанта 3 (Доставка їжі). Колекції призначені для зберігання даних про клієнтів, меню страв та історію замовлень.

```
> use lab5
< switched to db lab5
> db.createCollection("clients")
db.createCollection("dishes")
db.createCollection("orders")
< { ok: 1 }
> show collections
< clients
  dishes
  orders
lab5> |
```

2. Наповнення бази даних (CRUD: Create)

Для подальшої роботи було згенеровано та додано тестові дані. Використовувався метод insertMany для масового додавання документів. Було додано 10 клієнтів, 15 страв та 30 замовлень зі складною структурою (масиви товарів, дати, статуси).

```
> db.clients.insertMany([
  { name: "Олександр Іваненко", phone: "+380501112233", district: "Голосіївський" },
  { name: "Марія Петренко", phone: "+380671234567", district: "Печерський" },
  { name: "Дмитро Коваленко", phone: "+380639998877", district: "Оболонський" },
  { name: "Анна Шевченко", phone: "+380991112233", district: "Шевченківський" },
  { name: "Олег Бойко", phone: "+380503334455", district: "Дарницький" },
  { name: "Ірина Ткаченко", phone: "+380675556677", district: "Голосіївський" },
  { name: "Максим Кравченко", phone: "+380637778899", district: "Подільський" },
  { name: "Олена Мороз", phone: "+380992223344", district: "Святошинський" },
  { name: "Андрій Лисенко", phone: "+380504445566", district: "Дніпровський" },
  { name: "Вікторія Мельник", phone: "+380678889900", district: "Печерський" }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a05c7d1c022ebba7bdb045a'),
    '1': ObjectId('6a05c7d1c022ebba7bdb045b'),
    '2': ObjectId('6a05c7d1c022ebba7bdb045c'),
    '3': ObjectId('6a05c7d1c022ebba7bdb045d'),
    '4': ObjectId('6a05c7d1c022ebba7bdb045e'),
    '5': ObjectId('6a05c7d1c022ebba7bdb045f'),
    '6': ObjectId('6a05c7d1c022ebba7bdb0460'),
    '7': ObjectId('6a05c7d1c022ebba7bdb0461'),
    '8': ObjectId('6a05c7d1c022ebba7bdb0462'),
    '9': ObjectId('6a05c7d1c022ebba7bdb0463')
  }
}
```

```

> db.dishes.insertMany([
  { name: "Піца Маргарита", cuisine: "Італійська", price: 250, isVegan: true },
  { name: "Піца Пепероні", cuisine: "Італійська", price: 300, isVegan: false },
  { name: "Суші Філадельфія", cuisine: "Японська", price: 350, isVegan: false },
  { name: "Суші Каліфорнія", cuisine: "Японська", price: 320, isVegan: false },
  { name: "Бургер з яловичиною", cuisine: "Американська", price: 280, isVegan: false },
  { name: "Веганський бургер", cuisine: "Американська", price: 290, isVegan: true },
  { name: "Салат Цезар", cuisine: "Європейська", price: 200, isVegan: false },
  { name: "Грецький салат", cuisine: "Європейська", price: 180, isVegan: true },
  { name: "Том Ям", cuisine: "Азіатська", price: 380, isVegan: false },
  { name: "Рамен зі свининою", cuisine: "Азіатська", price: 310, isVegan: false },
  { name: "Паста Карбонара", cuisine: "Італійська", price: 270, isVegan: false },
  { name: "Стейк із лосося", cuisine: "Європейська", price: 450, isVegan: false },
  { name: "Борщ український", cuisine: "Українська", price: 150, isVegan: false },
  { name: "Вареники з картоплею", cuisine: "Українська", price: 130, isVegan: true },
  { name: "Млинці з сиром", cuisine: "Українська", price: 140, isVegan: false }
])
< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a05c802c022ebba7bdb0464'),
    '1': ObjectId('6a05c802c022ebba7bdb0465'),
    '2': ObjectId('6a05c802c022ebba7bdb0466'),
    '3': ObjectId('6a05c802c022ebba7bdb0467'),
    '4': ObjectId('6a05c802c022ebba7bdb0468'),
    '5': ObjectId('6a05c802c022ebba7bdb0469'),
    '6': ObjectId('6a05c802c022ebba7bdb046a'),
    '7': ObjectId('6a05c802c022ebba7bdb046b'),
    '8': ObjectId('6a05c802c022ebba7bdb046c'),
  },
}

```

```

> let ordersArray = [];
let statuses = ["Completed", "Pending", "Cancelled"];
let phones = ["+380501112233", "+380671234567", "+380639998877", "+380991112233"];

for (let i = 1; i <= 30; i++) {
  let orderNum = i < 10 ? "0" + i : i;
  ordersArray.push({
    orderNo: "ORD0" + orderNum,
    clientPhone: phones[i % 4],
    date: new Date(2024, 4, (i % 28) + 1),
    items: [
      { dish: "Піца Маргарита", qty: (i % 3) + 1, price: 250 },
      { dish: "Салат Цезар", qty: (i % 2) + 1, price: 200 }
    ],
    status: statuses[i % 3]
  });
}
db.orders.insertMany(ordersArray)
< {
  acknowledged: true,
}

```

3. Виконання запитів з операторами фільтрації (CRUD: Read)

Було розроблено серію запитів для вибірки даних за певними критеріями, використовуючи оператори порівняння та логічні зв'язки:

Запит 1: Використання оператора \$lt (менше ніж) Знайти всі страви, ціна яких строго менша за 200 грн.

```
> db.dishes.find({ price: { $lt: 200 } })
< {
  _id: ObjectId('6a05c802c022ebba7bdb046b'),
  name: 'Грецький салат',
  cuisine: 'Європейська',
  price: 180,
  isVegan: true
}
{
  _id: ObjectId('6a05c802c022ebba7bdb0470'),
  name: 'Борщ український',
  cuisine: 'Українська',
  price: 150,
  isVegan: false
}
{
  _id: ObjectId('6a05c802c022ebba7bdb0471'),
  name: 'Вареники з картоплею',
  cuisine: 'Українська',
  price: 130,
  isVegan: true
}
{
  _id: ObjectId('6a05c802c022ebba7bdb0472'),
  name: 'Млинці з сиром',
  cuisine: 'Українська',
  price: 140,
  isVegan: false
}
```

Запит 2: Використання оператора \$in (входження в список) Знайти клієнтів, які проживають або в Голосіївському, або в Печерському районах.

```
> db.clients.find({ district: { $in: ["Голосіївський", "Печерський"] } })
< {
  _id: ObjectId('6a05c7d1c022ebba7bdb045a'),
  name: 'Олександр Іваненко',
  phone: '+380501112233',
  district: 'Голосіївський'
}
{
  _id: ObjectId('6a05c7d1c022ebba7bdb045b'),
  name: 'Марія Петренко',
  phone: '+380671234567',
  district: 'Печерський'
}
{
  _id: ObjectId('6a05c7d1c022ebba7bdb045f'),
  name: 'Ірина Ткаченко',
  phone: '+380675556677',
  district: 'Голосіївський'
}
{
  _id: ObjectId('6a05c7d1c022ebba7bdb0463'),
  name: 'Вікторія Мельник',
  phone: '+380678889900',
  district: 'Печерський'
}
```

Запит 3: Використання оператора \$gt (більше ніж) для вкладених масивів Знайти замовлення, де кількість замовлених порцій будь-якої зі страв більша за 1.

```
> db.orders.find({ "items.qty": { $gt: 1 } })
< {
  _id: ObjectId('6a05c898c022ebba7bdb0473'),
  orderNo: 'ORD001',
  clientPhone: '+380671234567',
  date: 2024-05-01T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 2,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 2,
      price: 200
    }
  ],
  status: 'Pending'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb0474'),
  orderNo: 'ORD002',
  clientPhone: '+380639998877',
  date: 2024-05-02T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 3,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 1,
      price: 200
    }
  ],
  status: 'Cancelled'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb0475'),
  orderNo: 'ORD003',
  clientPhone: '+380991112233',
  date: 2024-05-03T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 1,
      price: 250
    },
  ],
}
```


Запит 4: Комбінована умова з \$or (АБО) Знайти страви, які або підходять для веганів, або належать до італійської кухні.

```
db.dishes.find({
  $or: [
    { isVegan: true },
    { cuisine: "Італійська" }
  ]
})
[
  {
    _id: ObjectId('6a05c802c022ebba7bdb0464'),
    name: 'Піца Маргарита',
    cuisine: 'Італійська',
    price: 250,
    isVegan: true
  },
  {
    _id: ObjectId('6a05c802c022ebba7bdb0465'),
    name: 'Піца Пеппероні',
    cuisine: 'Італійська',
    price: 300,
    isVegan: false
  },
  {
    _id: ObjectId('6a05c802c022ebba7bdb0469'),
    name: 'Веганський бургер',
    cuisine: 'Американська',
    price: 290,
    isVegan: true
  },
  {
    _id: ObjectId('6a05c802c022ebba7bdb046b'),
    name: 'Грецький салат',
    cuisine: 'Європейська',
    price: 180,
    isVegan: true
  },
  {
    _id: ObjectId('6a05c802c022ebba7bdb046e'),
    name: 'Паста Карбонара',
    cuisine: 'Італійська',
    price: 270,
    isVegan: false
  },
  {
    _id: ObjectId('6a05c802c022ebba7bdb0471'),
    name: 'Вареники з картоплею',
    cuisine: 'Українська',
    price: 130,
    isVegan: true
  }
]
```

Запит 5: Комбінована умова з \$and та операторами дат (\$gte, \$lte) Знайти всі успішні замовлення (Completed), які були оформлені з 1 по 15 травня 2024 року.

```
> db.orders.find({
  $and: [
    { status: "Completed" },
    { date: { $gte: new Date("2024-05-01"), $lte: new Date("2024-05-15") } }
  ]
})
< {
  _id: ObjectId('6a05c898c022ebba7bdb0475'),
  orderNo: 'ORD003',
  clientPhone: '+380991112233',
  date: 2024-05-03T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 1,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 2,
      price: 200
    }
  ],
  status: 'Completed'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb047b'),
  orderNo: 'ORD006',
  clientPhone: '+380639998877',
  date: 2024-05-06T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
      qty: 1,
      price: 250
    },
    {
      dish: 'Салат Цезар',
      qty: 1,
      price: 200
    }
  ],
  status: 'Completed'
}
{
  _id: ObjectId('6a05c898c022ebba7bdb047b'),
  orderNo: 'ORD009',
  clientPhone: '+380671234567',
  date: 2024-05-09T21:00:00.000Z,
  items: [
    {
      dish: 'Піца Маргарита',
```

1. Перевірка запиту ДО створення індексу Спочатку подивимося, як база даних шукає інформацію без індексу. Виконай цей запит:

```
> db.orders.find({ status: "Completed" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'lab5.orders',
    indexFilterSet: false,
    parsedQuery: {
      status: {
        '$eq': 'Completed'
      }
    },
    queryHash: '4B6FAB47',
    planCacheKey: '4B6FAB47',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'COLLSCAN',
      filter: {
        status: {
          '$eq': 'Completed'
        }
      },
      direction: 'forward'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 10,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 30,
    executionStages: {
      stage: 'COLLSCAN',
      filter: {
        status: {
          '$eq': 'Completed'
        }
      },
      nReturned: 10,
      executionTimeMillisEstimate: 0,
      works: 31,
      advanced: 10,
      needTime: 20,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 30
    }
  }
}
```

2. Створення індексів 3-й варіант (клієнти, страви, замовлення).

```
> db.clients.createIndex({ phone: 1 }, { unique: true })
< phone_1
> db.dishes.createIndex({ cuisine: 1, price: 1 })
< cuisine_1_price_1
> db.orders.createIndex({ status: 1, date: -1 })
< status_1_date_-1
lab5> |
```

3. Перевірка запиту ПІСЛЯ створення індексу

```
lab5> db.orders.find({ status: "Completed" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'lab5.orders',
    indexFilterSet: false,
    parsedQuery: {
      status: {
        '$eq': 'Completed'
      }
    },
    queryHash: '486FAB47',
    planCacheKey: '6SD76298',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: {
          status: 1,
          date: -1
        },
        indexName: 'status_1_date_-1',
        isMultiKey: false,
        multiKeyPaths: {
          status: [],
          date: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          status: [
            '["Completed", "Completed"]'
          ],
          date: [
            '[MaxKey, MinKey]'
          ]
        }
      },
      rejectedPlans: []
    },
    executionStats: {
      executionSuccess: true,
      nReturned: 10,
      executionTimeMillis: 15,
      totalKeysExamined: 10,
    }
  }
}
```

4. Перегляд та видалення індексів Щоб переглянути всі існуючі індекси в колекції orders

```
> db.orders.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { status: 1, date: -1 }, name: 'status_1_date_-1' }
]
> db.orders.dropIndex("status_1_date_-1")
< { nIndexesWas: 2, ok: 1 }
lab5>
```

Висновок

У ході виконання лабораторної роботи я ознайомився з основами роботи в NoSQL БД MongoDB. На практиці було опановано використання інтерфейсу MongoDB Compass та оболонки mongosh.

Я навчився проектувати структуру документів з гнучкою схемою, виконувати базові CRUD-операції та створювати складні запити з використанням фільтрів порівняння (\$gt, \$lt, \$in) та логічних операторів. Особливу увагу було приділено оптимізації запитів: за допомогою інструменту explain я переконався, що правильно налаштовані індекси дозволяють базі даних працювати швидше, уникаючи повного сканування колекції (COLLSCAN). Отримані навички є базовими для розробки сучасних масштабованих додатків, що використовують неструктуровані дані.